

# Intermediate (test a tool without calling the LLM)

## — Hints

---

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper pytest, including how a tool fails and what it does with bad arguments). Read Basic first even if you already know pytest — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Hint 1 — The idea, and the exact pieces

### Basic Version

A "tool" from `06_tools_function_calling` is just a normal Python function underneath — the model only decides *when* to call it. The function's own logic (what it does once called) doesn't need a model at all to test.

So: stop thinking about the LLM completely for this exercise. Pick one tool from Doc06's `tools.py`, call it directly yourself with made-up arguments, and check what it returns — the same way you'd test any other function.

One detail: Doc06's tools are wrapped with LangChain's `@tool`, so you call them the same way Doc06's `tool_harness.py` did — `add.invoke({"a": 2, "b": 3})`, with the arguments in a dict — not `add(2, 3)`.

### Intermediate Version

This exercise separates two questions that get blurred together if you only ever test through the full agent:

1. Does the tool itself do the right thing, given specific arguments?
2. Does the model choose to call it, with the right arguments, at the right time?

Question 1 has nothing to do with the LLM, so it gets a normal, fast, exact pytest test — no API key, no network, no cost, no flakiness. Question 2 is a separate, harder problem (closer to the Real-world exercise later in this document).

A tool test isn't only "does it work with good input." A real model sometimes calls a tool in a way that fails, or with arguments that make no sense. So also test: what does the tool return when it fails, and what happens when the arguments have the wrong type?

The exact pieces:

- Copy Doc06's `tools.py` into `practice/`, with the files it imports (`http_client.py`, `exceptions.py`, `logging_setup.py`), unchanged.
- `from tools import add, flaky_lookup` — `add` is pure math, and `flaky_lookup` has a `should_fail` switch built in for exactly this kind of test.
- `flaky_lookup.invoke({"query": "x", "should_fail": True})` — the tool catches its own error and returns an `"Error: ..."` string the model can read, instead of crashing the agent.

- `with pytest.raises(ValidationError):` — checks that a block of code raises that error. `@tool` checks the arguments against the function's type hints (with Pydantic, like Doc04/06), so `add.invoke({"a": "two", "b": 3})` raises `ValidationError` before `add` even runs. Import it with `from pydantic import ValidationError`.
- Skip `get_weather` here: it calls a real weather API over the network, so it isn't a no-network test.

**Difference between Basic and Intermediate:** Basic calls one tool directly and checks what comes back. Intermediate separates "does the tool work" from "does the model call it," and adds the two failure checks a real tool needs: an error result the model can read, and a clean rejection of wrongly typed arguments.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Hint 2 — The plan, and almost the whole thing

### Basic Version

```
import the tool directly (not through the model)

def test_add():
    result = add.invoke with a = 2, b = 3
    assert result is exactly 5
```

Here's almost the whole thing — just try running it and reading it line by line:

```
# practice/tool_test_no_llm_practice.py
from tools import add

def test_add():
    result = add.invoke({"a": 2, "b": 3})
    assert result == 5
```

Run it from inside `practice/` with `pytest tool_test_no_llm_practice.py -v`.

### Intermediate Version

```
from tools import add, flaky_lookup

def test_add_returns_the_sum():    a couple of exact sums
def test_flaky_lookup_success():   no should_fail -> "Result for ..."
def test_flaky_lookup_failure():  should_fail=True -> "Error: ..."
def test_add_rejects_wrong_type():
    with pytest.raises(ValidationError):
        add.invoke with a = "two"
```

Here's most of it — write the failure test yourself:

```
# practice/tool_test_no_llm_practice.py
import pytest
from pydantic import ValidationError
from tools import add, flaky_lookup
```

```
def test_add_returns_the_sum():
    assert add.invoke({"a": 2, "b": 3}) == 5
    assert add.invoke({"a": -1, "b": 1}) == 0

def test_flaky_lookup_success():
    result = flaky_lookup.invoke({"query": "refund policy"})
    assert result == "Result for refund policy"

def test_flaky_lookup_failure():
    # your turn: call it with should_fail=True, and check the exact
    # "Error: ..." string it returns instead of crashing
    ...

def test_add_rejects_wrong_type():
    with pytest.raises(ValidationError):
        add.invoke({"a": "two", "b": 3})
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)